Enumerative Program Synthesis with Observational Equivalence Reduction

Varun Varma

Governor's School at Innovation Park

May 11th, 2021

Acknowledgements

I would like to thank Dr. Ales Psaker for his guidance throughout the research process.

Abstract

Program synthesis is the problem of generating a program that satisfies a given specification. Enumerative program synthesis is a powerful type of program synthesis that iterates through the search space of possible programs to find a solution. The main issue with enumerative program synthesis is that the search space explodes in size for harder synthesis problems. One way of optimizing enumerative synthesis is through equivalence reduction. Enumerative synthesis generates many complete programs that have the same outputs (for example, $x + 0$ and x). Equivalence reduction identities such equivalent programs and removes them from the search space, which drastically reduces the number of programs that need to be searched. This research project implemented an enumerative synthesis engine that uses observational equivalence reduction, a form of equivalence reduction that compares program outputs to determine equivalence. The synthesis engine takes a set of input-output integer pairs and searches for a program that has the same input-outputs. The synthesizer was tested on a set of nine synthesis problems from the book *Hacker's Delight* with observational equivalence reduction enabled and disabled. On all benchmarks, the synthesizer completed synthesis far faster with observational equivalence reduction enabled. Furthermore, observational equivalence reduction gave more percentage efficiency gains the harder the synthesis problem was, which suggests that observational equivalence reduction can scale to be useful for large synthesis programs rather than just toy problems. This research shows that observational equivalence reduction can be used to optimize industry quality program synthesis engines, which will help accelerate and democratize programming. Further work could explore equivalence reduction through other means, such as e-graphs.

Enumerative Program Synthesis with Observational Equivalence Reduction

Program synthesis is the problem of generating a program that satisfies a given specification. Sufficiently scaled and deployed program synthesis could free programmers from low-level code by allowing all programs to be described by high-level specifications, which would accelerate programming speed and make high-quality code accessible to anyone. While the field of program synthesis is still in its early stages and even the most advanced synthesis engines can only solve toy problems, new optimizations are aggressively improving the state of the art. One such optimization is applying observational equivalence reduction to enumerative synthesis.

Enumerative program synthesis is a type of program synthesis where the search space is iterated until a solution is found. Enumerative program synthesis has seen more interest than other, more symbolic, approaches because it allows for heavy optimizations through search algorithms and heuristics (Alur, Singh, Fisman, & Solar-Lezama, 2018). The main problem with enumerative program synthesis is that the search space explodes in size or becomes infinite for larger programs, so these optimizations are often mandatory for harder synthesis problems. A type of enumerative synthesis is bottom-up enumerative synthesis, which starts with zero-arity programs and then generates new programs by substituting arguments of non-zero-arity terms with previously generated programs. For example, the program $x + x$ can be generated with the zero-arity program x and the two-arity term $+$. A useful property of this approach is that each newly generated program is complete and can be evaluated for its outputs. Another important property of bottom-up enumerative synthesis is that it tends to generate many equivalent programs, especially when the language has commutive terms. (Barke, Peleg, & Polikarpova,

2020). These properties make bottom-up enumerative synthesis a prime candidate for optimization via equivalence reduction.

Equivalence reduction identifies equivalent programs and removes all but the most efficient one from the search space. For example, x and $x + 0$ are equivalent programs, but x is the most simplified form so $x + 0$ can be discarded. Because the search space of bottom-up enumerative synthesis explodes as height increases, reductions in the lower heights eliminate a massive part of the search space. (Smith & Albarghouthi, 2019). There are various methods to detect the equivalence of two programs. A symbolic approach to equality reduction is to encode programs into e-graphs and deduce equality. However, such an approach requires formal specifications of the target language. (Willsey et al., 2021). Modern program synthesis tasks only supply the target language's grammar, so symbolic approaches to equivalence reduction are limited in their practicality (Alur et al., 2013). An alternative is observational equivalence reduction, where the outputs of programs are compared to determine equality. This approach works well because bottom-up enumerative synthesis always generates evaluable programs whose output can be computed.

Enumerative synthesis is a powerful type of program synthesis that is applicable for aggressive optimizations. By conducting enumerative synthesis with a bottom-up approach, the synthesis engine will generate complete but often equal programs. Observational equivalence reduction can optimize enumerative synthesis by using the evaluated outputs of bottom-up generated programs to identify and remove these equivalent programs.

Question and Hypothesis

The research asks if observational equivalence reduction optimizes enumerative program synthesis. The hypothesis was that the enumerative program synthesis engine will find solutions to a set of problems faster with observational equivalence reduction enabled than disabled because observational equivalence reduction will reduce the size of the search space.

Materials and Methods

A bottom-up enumerative program synthesis engine with observational equivalence reduction (outlined in Figure 1) was benchmarked on nine bit-manipulation problems adapted from the *Hacker's Delight* book (displayed in Figure 2). Each problem was defined by 256 input-out pairs calculated with the target program. The synthesizer inputs a specification of type $[(i8, i8); 256]$ and outputs the total number of programs iterated and a function of type $i8 \rightarrow i8$ that satisfies the specification. The synthesizer was evaluated on each problem in the benchmark, once with observational equivalence reduction enabled and once with it disabled. The total synthesis time, the size of synthesized program, and the number of programs iterated were recorded into a .csv file. The synthesis engine was written in Rust 1.53-nightly. The code can be found at https://github.com/varranvar/enumerative_synthesis_with_obe. All benchmarks were run on an Intel Core i7-9750H processer with 32 GB of RAM on a single thread.

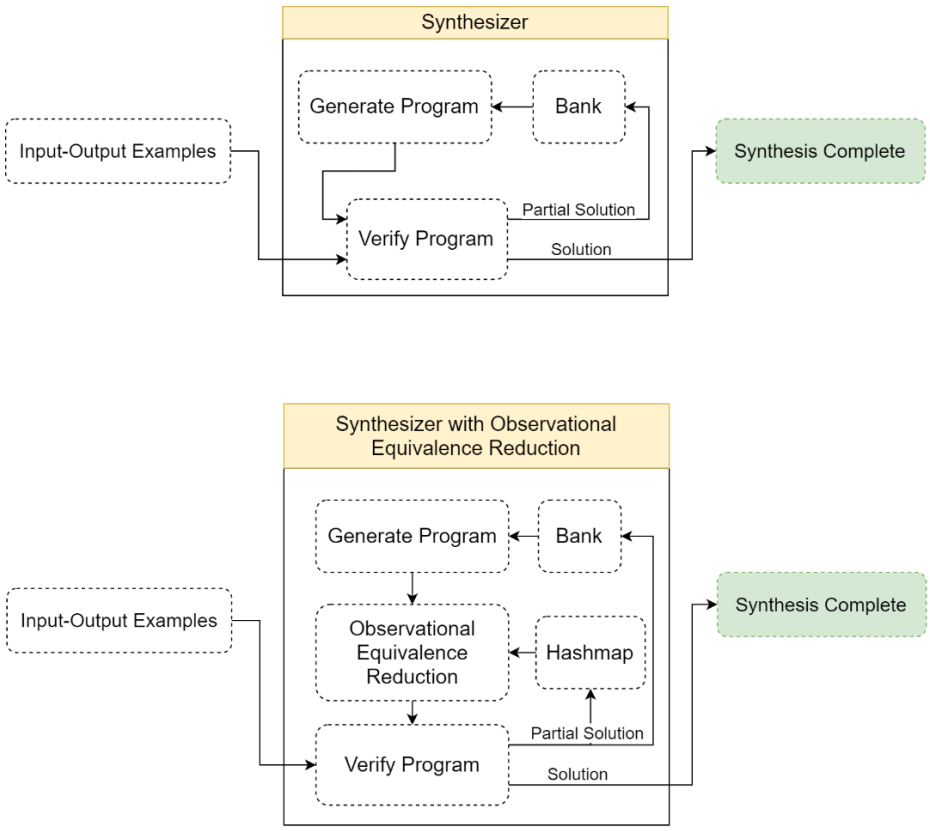

Figure 1. Overview of the bottom-up enumerative synthesis engine with observational equivalence reduction enabled and disabled.

Hacker's Delight Bit Manipulation Programs 1 - 9		
P1(x): Turn off rightmost bit 1. `a = x - 1` 2. `result = x & a`	**P2**(x): Test whether an unsigned integer is of the form 2^{n-1} 1. `a = x + 1` 2. `result = x & a`	**P3**(x): Isolate the rightmost bit 1. `a = -x` 2. `result = x & a`
P4(x): Form a mask that identifies the rightmost bit and the trailing zeros 1. `a = x – 1` 2. `result = x ^ a`	**P5**(x): Right propagate rightmost bit 1. `a = x – 1` 2. `result = x | a`	**P6**(x): Turn on the right-most zero-bit in a word 1. `a = x + 1` 2. `result = x | a`
P7(x): Isolate the rightmost zero-bit 1. `a = !x` 2. `b = x + 1` 3. `result = a & b`	**P8**(x): Form a mask that identifies the trailing zeros 1. `a = x – 1` 2. `b = !x` 3. `result = a & b`	**P9**(x): Absolute value function 1. `a = x >> max` 2. `b = x | a` 3. `result = b - a`

Figure 2: The nine benchmark problems from the *Hacker's Delight* problem set.

Results

Problem	Synthesis Time (μs) Without Observational Equivalence Reduction	Synthesis Time (μs) With Observational Equivalence Reduction
1	47439	11745
2	11421	6040
3	592128	46845
4	1179776	83811
5	7507	4082
6	2898	2087
7	599882	48557
8	1725702	43172
9	4330286	256931

Figure 3. A table of synthesis times of nine synthesis problems with observational equivalence reduction enabled and disabled.

Problem	Synthesized Program Without Observational Equivalence Reduction	Synthesized Program With Observational Equivalence Reduction
1	(& (- x 1) (+ x 0))	(& (- x 1) x)
2	(& (^ x 0) (+ 1 x))	(& x (+ x 1))
3	(& (/ x 0) (- 0 x))	(& x (- 0 x))
4	(^ (- 0 x) (! x))	(^ (+ x -128) (+ 127 x))
5	(| (| 1 x) (- x 1))	(| (| x 1) (- x 1))
6	(| (| 1 x) (+ 1 x))	(| (| x 1) (+ x 1))
7	(& (+ x 1) (! x))	(& (+ 1 x) (! x))
8	(& (- x 1) (! x))	(& (! x) (- x 1))
9	(& (^ (- 0 x) (^ 1 x)) (/ (- 0 x) (>> x 0)))	(& (^ (- 0 x) (^ x 1)) (/ (- 0 x) x))

Figure 4. A table of synthesized programs of nine synthesis problems with observational equivalence reduction enabled and disabled.

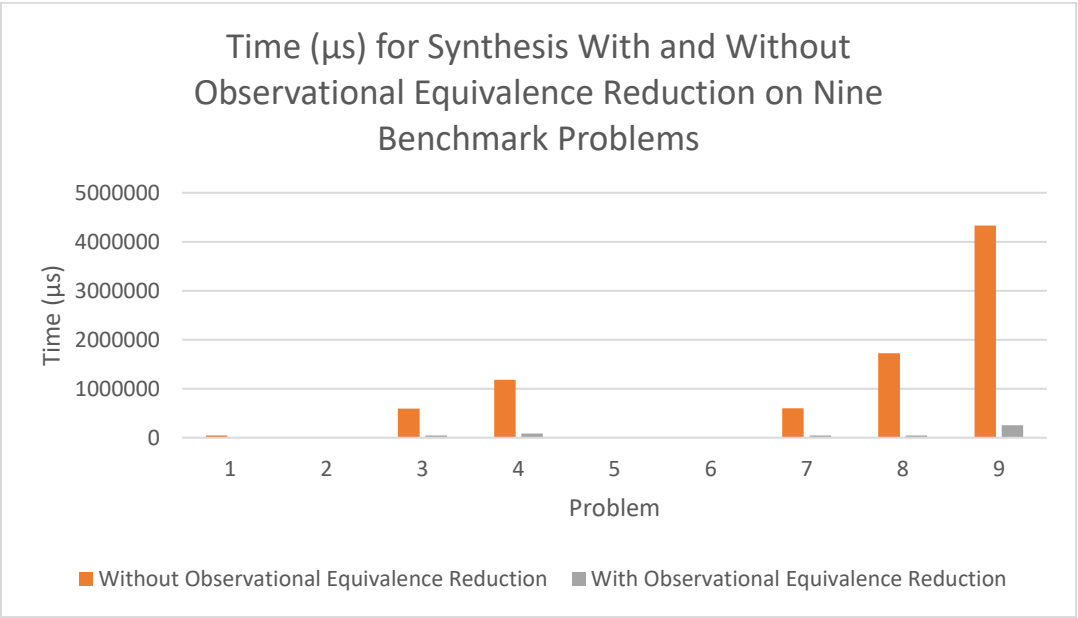

Figure 5. A graph displaying the synthesis time of nine synthesis problems with observational equivalence reduction enabled and disabled.

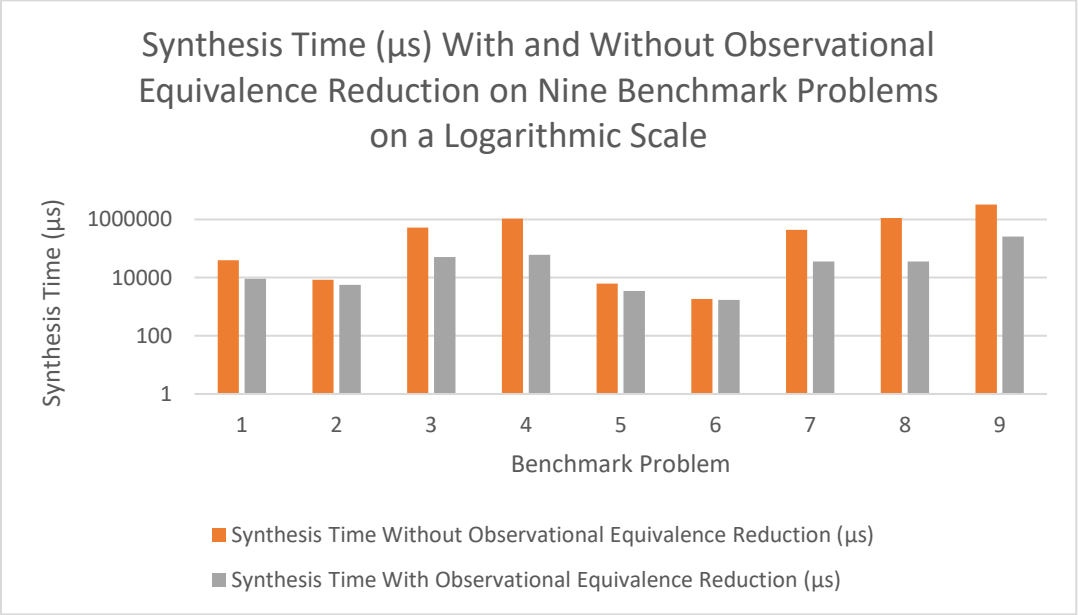

Figure 6. A graph displaying the synthesis time of nine synthesis problems with observational equivalence reduction enabled and disabled on a logarithmic scale.

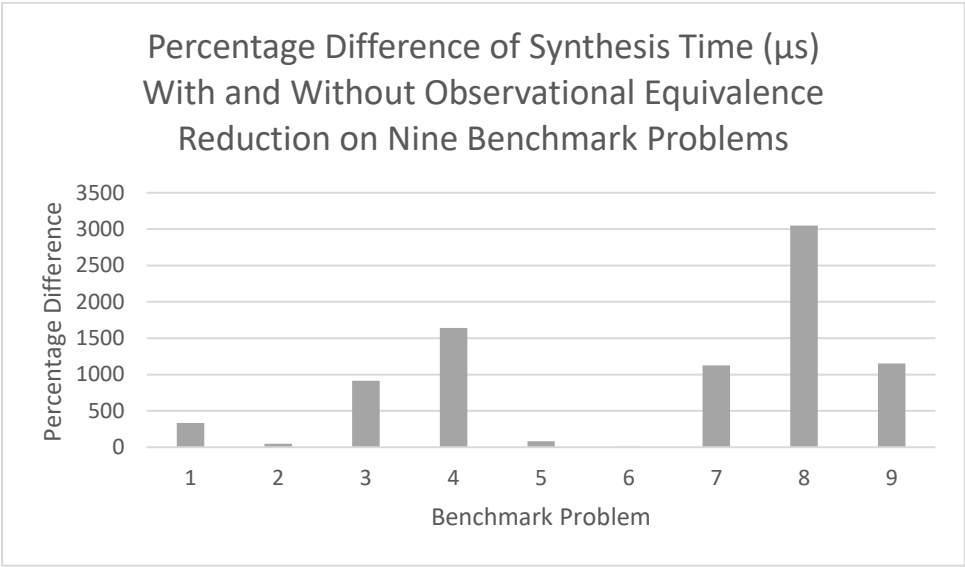

Figure 7. A graph displaying the percentage difference in synthesis time between the benchmarks with observational equivalence reduction enabled and the benchmarks with it disabled.

Data Analysis

The synthesis times were recorded in Figure 3 and plotted in Figure 5. Synthesis with equivalence reduction enabled outperforms synthesis with equivalence reduction disabled on every problem. Problem 6 is a slight outlier with a percentage difference of only 6.86%. This can be attributed to a solution being found with a low number of iterations where there is less room for optimization from observational equivalence reduction. The synthesis times of the problems have a large range of 3228994 microseconds, which is expected because the problems vary wildly in difficulty. To display this large range of synthesis times, the data was replotted on a logarithmic scale in Figure 3. Figure 6 displays the synthesized programs from each of the benchmarks in post-fix format. Equivalence reduction is apparent in these programs because the equivalence reduced programs have only simplified terms. For example, in Problem 1 the term *(+ x 0)* is replaced with the smaller yet equivalent term *x*. Figure 7 displays the percentage difference between the two benchmark sets. A higher percentage difference suggests higher optimization. Higher percentage difference also corresponded with higher synthesis time.

Conclusion

On all benchmarks, the synthesizer completes the problem faster with observational equivalence reduction enabled. Through the clear efficiency gains, the results suggest that observational equivalence reduction dramatically reduces the synthesis time of enumerative bottom-up program synthesis. In addition, comparing synthesis time with percentage difference in synthesis time shows that higher efficiency gains corresponded with problems that took more time, which suggests that observational equivalence reduction becomes more effective for more difficult problems. The results support the hypothesis that the enumerative program synthesis engine will find solutions to a set of problems faster with observational equivalence reduction enabled than disabled. In addition, the results suggest that observational equivalence reduction can scale to be useful for real-world problems, rather than just toy examples, because it becomes more useful the harder the synthesis problem is. This research can be used to optimize industry-standard program synthesis engines, which will help accelerate and democratize programming. Possible sources of error in this research include compilation differences and hidden processes affecting the benchmarking. Further research could explore alternate methods of equivalence reduction, such as encoding programs into e-graphs, or testing observational equivalence reduction on heuristic guided synthesis engines.

References

Alur, R., Bodik, R., Juniwal, G., Martin, M. M., Raghothaman, M., Seshia, S. A., . . . Udupa, A. (2013). Syntax-guided synthesis. *2013 Formal Methods in Computer-Aided Design*. doi:10.1109/fmcad.2013.6679385

Alur, R., Singh, R., Fisman, D., & Solar-Lezama, A. (2018). Search-based program synthesis. *Communications of the ACM, 61*(12), 84-93. doi:10.1145/3208071

Barke, S., Peleg, H., & Polikarpova, N. (2020). Just-in-time learning for bottom-up enumerative synthesis. *Proceedings of the ACM on Programming Languages, 4*(OOPSLA), 1-29. doi:10.1145/3428295

Smith, C., & Albarghouthi, A. (2019). Program synthesis with equivalence reduction. *Lecture Notes in Computer Science,* 24-47. doi:10.1007/978-3-030-11245-5_2

Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P. (2021). Egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages, 5*(POPL), 1-29. doi:10.1145/3434304